AWS Data Platform DevOps - short Python code reviews

These are verbal exercises. You do not need to open an editor, write a complete function, implement tests, or use an AWS SDK. Please explain your reasoning in English. Spend about four minutes on exercise 1, two minutes on exercise 2, and four minutes discussing your actual automation experience.

Exercise 1 - failed job names

The following records represent processing jobs:

```
jobs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "FAILED"},
]
```

Each call should return only the failed job names from that call, independently of earlier calls. This function has two bugs:

```
def failed_jobs(jobs, failed=[]):
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
        return failed
```

Questions

1. What does the function return for the sample input, and why?
2. What small change would make it examine every job before returning?
3. After making only that change, what would two successive calls to `failed_jobs(jobs)` return for the sample input? Assume a fresh function definition and no other calls. Explain the second bug and how to fix it.
4. After fixing both bugs, what should it return for an empty input and when no jobs have failed?

You may explain both corrections verbally; typing is optional.

Assume each record contains valid `name` and `status` fields. You do not need to design validation, retries, logging, or a monitoring system for this snippet.

Exercise 2 - jobs to retry

The following records represent completed processing runs:

```
runs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "TIMED_OUT"},
    {"name": "load_prices", "status": "CANCELLED"},
]
```

Only jobs with status `FAILED` or `TIMED_OUT` should be selected for retry. The function below does not follow that rule:

```
def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] == "FAILED" or "TIMED_OUT":
            retryable.append(job["name"])
    return retryable
```

Questions

1. What does `retryable_jobs(runs)` return, and what should it return?
2. Why does this condition also select successful or cancelled jobs?
3. How would you correct the condition? Explain verbally; typing is optional.
4. Which simple inputs would you check to confirm the correction works?

Assume every record contains valid `name` and `status` fields. Only explain the selection logic; no retry mechanism, AWS integration, or written tests are required.

Short experience discussion - 4 minutes

After both exercises, describe one Python automation you personally worked on: what it did, how failures were handled, and how you checked it before deployment. No implementation is required during this discussion.